گزارش پروژه: تشخیص علائم راهنمایی و رانندگی با شبکه عصبی

پیاده سازی در Google Colab

تهیه کننده : [محمد امین حری فراهانی]

درس: هوش مصنوعی — پروژه پایانی

تاریخ: ۳۱ دسامبر ۲۰۲۵

چکیده

فرآیند کامل طراحی، پیاده‌سازی و ارزیابی یک سامانه تشخیص علائم راهنمایی و رانندگی . در این پروژه از دیتاست **GTSRB** استفاده شده و یک شبکه عصبی کانولوشنی ساده با چند لایه کانولوشن و دراپ‌اوت برای طبقه‌بندی ۴۳ کلاس علامت‌ها آموزش داده شده است. مراحل شامل آماده‌سازی داده‌ها، آموزش مدل در محیطGoogle Colab ، بررسی نتایج با نمودارهای دقت/خطا و ماتریس اغتشاش، و نمایش تست یک تصویر واقعی می‌باشد.

فهرست مطالب

۱ مقدمه ۳

۲ دیتاست ۳

۱.۲ معرفی دیتاست ۳

۲.۲ ساختار فایل‌ها ۳

۳.۲ نحوه آوردن دیتا به Google Colab ۳

۳ پیش‌پردازش و آماده‌سازی داده ۴

۴ معماری شبکه ۵

۱.۴ بلوک‌های کانولوشن ۵

۵ فرآیند آموزش ۶

۶ نتایج ۸

۱.۶ ماتریس درهم‌ریختگی و گزارش طبقه‌بندی ۹

۲.۶ تحلیل Classification Report ۱۱

۷ نمونه تست یک تصویر واقعی ۱۲

۱.۷ محدودیت‌ها ۱۲

۲.۷ دسترسی به پروژه ۱۴

۱ مقدمه

تشخیص علائم راهنمایی و رانندگی یکی از مسائل مهم در حوزه بینایی ماشین است. هدف این پروژه توسعه یک مدل طبقه‌بندی تصویر است که قادر باشد نوع یک علامت جاده‌ای را از تصویر ورودی تشخیص دهد. در این پروژه تلاش شد مدلی پیاده‌سازی شود که هم قابل تحلیل باشد و هم عملکرد خوبی داشته باشد.

۲ دیتاست

۱.۲ معرفی دیتاست

دیتاست مورد استفاده در این پروژه **German Traffic Sign Recognition Benchmark (GTSRB)** است که شامل تصاویر رنگی از علائم راهنمایی و رانندگی آلمان می‌باشد. لینک دیتاست :

Kaggle k

۲.۲ ساختار فایل‌ها

نسخه‌ای که در این پروژه استفاده شده ساختار پوشه‌ای زیر را دارد:

- `Train/0/`, `Train/1/`, ..., `Train/42/` ـــ پوشه‌های کلاس‌های آموزشی
- `Test/` ـــ تصاویر تست

۳.۲ نحوه آوردن دیتا به Google Colab

برای آوردن دیتاست به محیط Colab ازGoogle Drive استفاده شد. مراحل کلی:

۱. اتصال Drive: `drive.mount('/content/drive')`

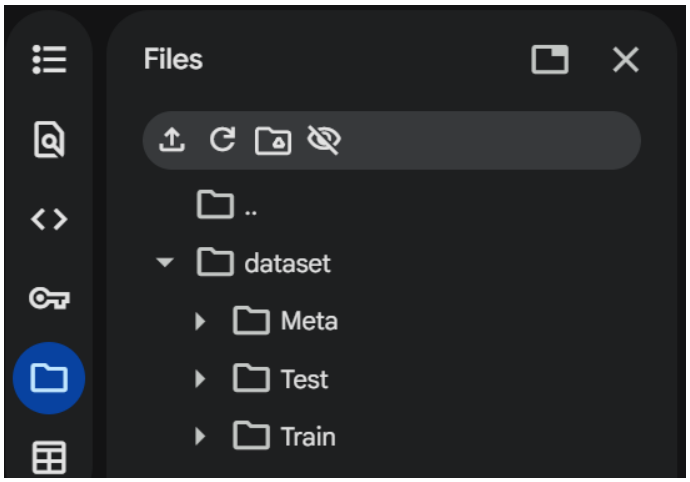
۲. باز کردن فایل zip و استخراج در مسیر `/content/dataset`

۳. بررسی ساختار با `os.listdir('/content/dataset')`

کد:

```
from google.colab import drive
drive.mount('/content/drive')

!unzip "/content/drive/MyDrive/archive.zip" -d "/content/dataset"
import os
print(os.listdir("/content/dataset"))
```



شکل ۱: فولد ها در کولب

۳ پیش‌پردازش و آماده‌سازی داده

مراحل پیش‌پردازش شامل موارد زیر است:

- تبدیل فضای رنگی BGR (OpenCV) به RGB
- Resize تصاویر به اندازه 30×30 پیکسل (برای سرعت و هماهنگی با معماری مدل)
- نرمال‌سازی پیکسل‌ها (تقسیم بر 0.255)
- تقسیم مجموعه به آموزش و اعتبارسنجی (حدود ۲۰% اعتبارسنجی)

کد :

```
image_data = []
image_labels = []

for i in range(NUM_CATEGORIES):
    path = os.path.join(train_path, str(i))
    images = os.listdir(path)
    for img_name in images:
        try:
            img_path = os.path.join(path, img_name)
            image = cv2.imread(img_path)
            image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
            image = cv2.resize(image, (IMG_WIDTH, IMG_HEIGHT))
            image_data.append(image)
            image_labels.append(i)
        except Exception as e:
            print(f"Error loading image {img_name}: {e}")
```

۴ معماری شبکه

برای تشخیص علائم راهنمایی و رانندگی، از یک **شبکه عصبی کانولوشنی (Convolutional Neural Network – CNN)** استفاده شده است. ورودی شبکه تصاویر رنگی با ابعاد $30 \times 30 \times 3$ است که پس از پیش‌پردازش (تغییر اندازه و نرمال‌سازی)، وارد شبکه می‌شوند. ابعاد کوچک تصویر باعث کاهش هزینه‌ی محاسباتی و افزایش سرعت آموزش شده، در حالی که اطلاعات کافی برای تشخیص علائم حفظ می‌شود.

۱.۴ بلوک‌های کانولوشن

شبکه شامل دو بلوک کانولوشنی متوالی است. هر بلوک از دو لایه کانولوشن، یک لایه Pooling Max و یک لایه Dropout تشکیل شده است.

در بلوک اول، از **۳۲ فیلتر کانولوشنی با اندازه‌ی کرنل** 5×5 استفاده شده است. این لایه‌ها وظیفه‌ی استخراج ویژگی‌های سطح پایین تصویر مانند لبه‌ها، گوشه‌ها و الگوهای ساده‌ی رنگی را بر عهده دارند. استفاده از کرنل نسبتاً بزرگ‌تر در این مرحله باعث می‌شود شبکه بتواند ساختارهای کلی‌تری از تصویر را تشخیص دهد.

پس از آن، لایه‌ی **Pooling Max با اندازه‌ی** 2×2 اعمال می‌شود که باعث کاهش ابعاد فضایی ویژگی‌ها و افزایش مقاومت مدل نسبت به تغییرات جزئی مکان (Translation Invariance) می‌گردد. سپس لایه‌ی **Dropout با نرخ 25.0** اضافه شده است تا با غیرفعال‌سازی تصادفی برخی نورون‌ها در زمان آموزش، از بیش‌برازش (Overfitting) جلوگیری شود.

در بلوک دوم، تعداد فیلترها به **۶۴ فیلتر با کرنل** 3×3 افزایش یافته است. در این مرحله، شبکه به استخراج ویژگی‌های سطح بالاتر مانند شکل کلی علائم، نمادها و ترکیب رنگ‌ها می‌پردازد. کوچک‌تر شدن اندازه‌ی کرنل به شبکه اجازه می‌دهد جزئیات دقیق‌تری را مدل‌سازی کند.

کد :

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPool2D, Dense, Flatten, Dropout

model = Sequential()
model.add(Conv2D(filters=32, kernel_size=(5,5), activation='relu', input_shape
    =(IMG_HEIGHT, IMG_WIDTH, CHANNELS)))
model.add(Conv2D(filters=32, kernel_size=(5,5), activation='relu'))
model.add(MaxPool2D(pool_size=(2, 2)))
model.add(Dropout(rate=0.25))

model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
model.add(MaxPool2D(pool_size=(2, 2)))
model.add(Dropout(rate=0.25))

model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(rate=0.5))
model.add(Dense(NUM_CATEGORIES, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['
```

```
accuracy'])
```

توضیح اجزاء:

- `Conv2D`: استخراج ویژگی‌های محلی تصویر (لبه‌ها، بافت‌ها)
- `MaxPool2D`: کاهش ابعاد ویژگی و مقاوم‌سازی نسبت به جابجایی
- `Dropout`: جلوگیری از بیش‌برازش (overfitting)
- `Flatten` و `Dense`: تبدیل ویژگی‌ها به یک بردار و تصمیم‌گیری نهایی

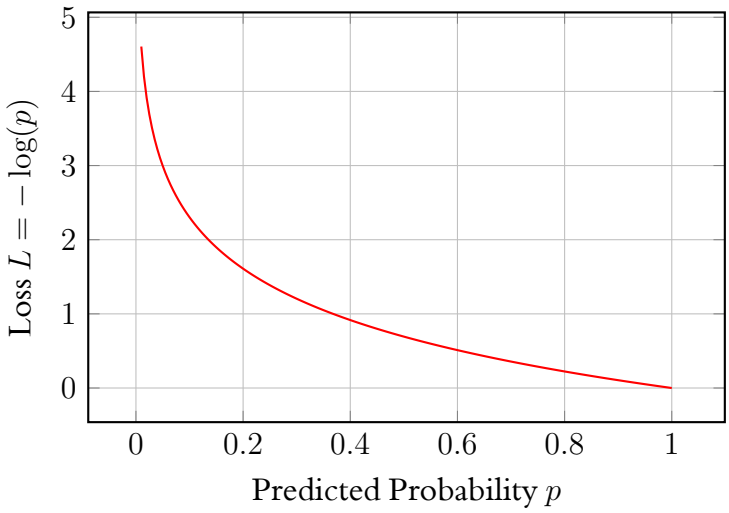
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	2,432
conv2d_1 (Conv2D)	(None, 22, 22, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 11, 11, 32)	0
dropout (Dropout)	(None, 11, 11, 32)	0
conv2d_2 (Conv2D)	(None, 9, 9, 64)	18,496
conv2d_3 (Conv2D)	(None, 7, 7, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 3, 3, 64)	0
dropout_1 (Dropout)	(None, 3, 3, 64)	0
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 256)	147,712
dropout_2 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 43)	11,051

شکل ۲: معماری شبکه عصبی کانولوشنی استفاده‌شده برای تشخیص علائم راهنمایی و ابعاد خروجی و پارامترهای شبکه

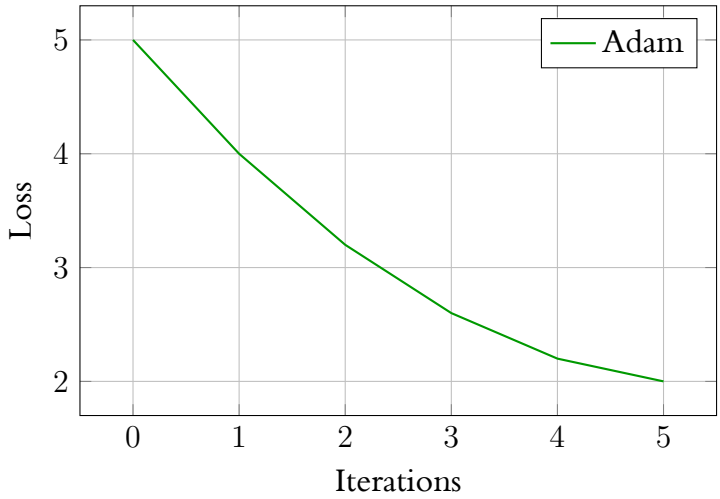
۵ فرآیند آموزش

برای آموزش، از تنظیمات زیر استفاده شد:

- تعداد: 10 epoch
- batch size: 32
- تابع هزینه: `categorical_crossentropy`
- بهینه‌ساز: `adam`



شکل ۳: نمودار تابع هزینه Cross-Entropy Categorical برای کلاس



کد آموزش:

```
epochs = 10
history = model.fit(X_train, y_train, batch_size=32, epochs=epochs,
    validation_data=(X_val, y_val))
```

برای ذخیره مدل:

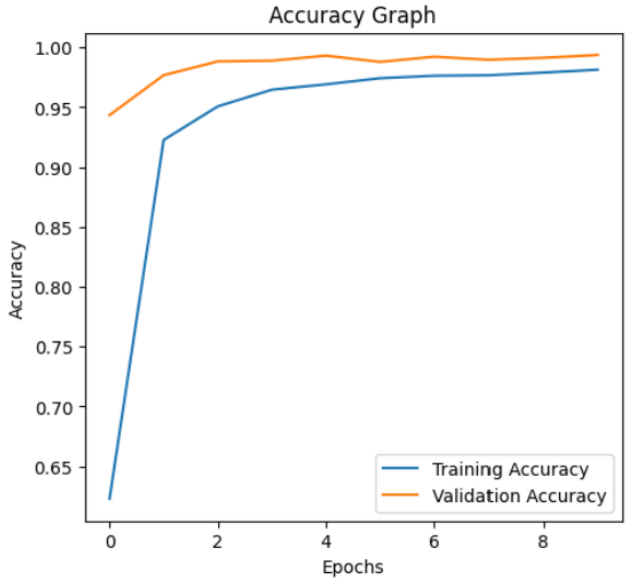
```
model.save("/content/drive/MyDrive/traffic_classifier.h5")
```

```
Epoch 1/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 134s 133ms/step - accuracy: 0.3678 - loss: 2.3397 - val_accuracy: 0.9434
Epoch 2/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 143s 134ms/step - accuracy: 0.9104 - loss: 0.3114 - val_accuracy: 0.9768
Epoch 3/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 132s 135ms/step - accuracy: 0.9464 - loss: 0.1749 - val_accuracy: 0.9883
Epoch 4/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 139s 132ms/step - accuracy: 0.9631 - loss: 0.1221 - val_accuracy: 0.9889
Epoch 5/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 129s 132ms/step - accuracy: 0.9698 - loss: 0.0981 - val_accuracy: 0.9931
Epoch 6/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 134s 136ms/step - accuracy: 0.9740 - loss: 0.0883 - val_accuracy: 0.9879
Epoch 7/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 130s 132ms/step - accuracy: 0.9744 - loss: 0.0850 - val_accuracy: 0.9922
Epoch 8/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 142s 132ms/step - accuracy: 0.9778 - loss: 0.0703 - val_accuracy: 0.9897
Epoch 9/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 129s 132ms/step - accuracy: 0.9778 - loss: 0.0711 - val_accuracy: 0.9913
Epoch 10/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 130s 132ms/step - accuracy: 0.9813 - loss: 0.0655 - val_accuracy: 0.9936
```

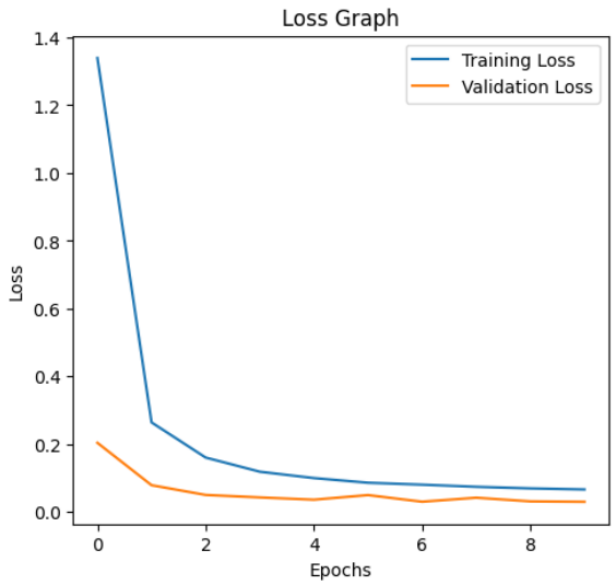
شکل ۴: نتایج آموزش مدل CNN: دقت و خطای مدل بر روی داده‌های آموزش و اعتبارسنجی در طول ۱۰ ایپاک. مشاهده می‌شود که مدل به سرعت همگرا شده و دقت اعتبارسنجی بالا و پایدار است.

۶ نتایج

در این بخش نتایج آموزش و ارزیابی مدل ارائه شده :



شکل ۵: نمودار دقت (Training / (Validation



شکل ۶: نمودار خطا (Training / (Validation

کد نمایش نمودار ها :

```
# Accuracy & Loss plots
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.savefig('accuracy_loss.png', bbox_inches='tight')
```

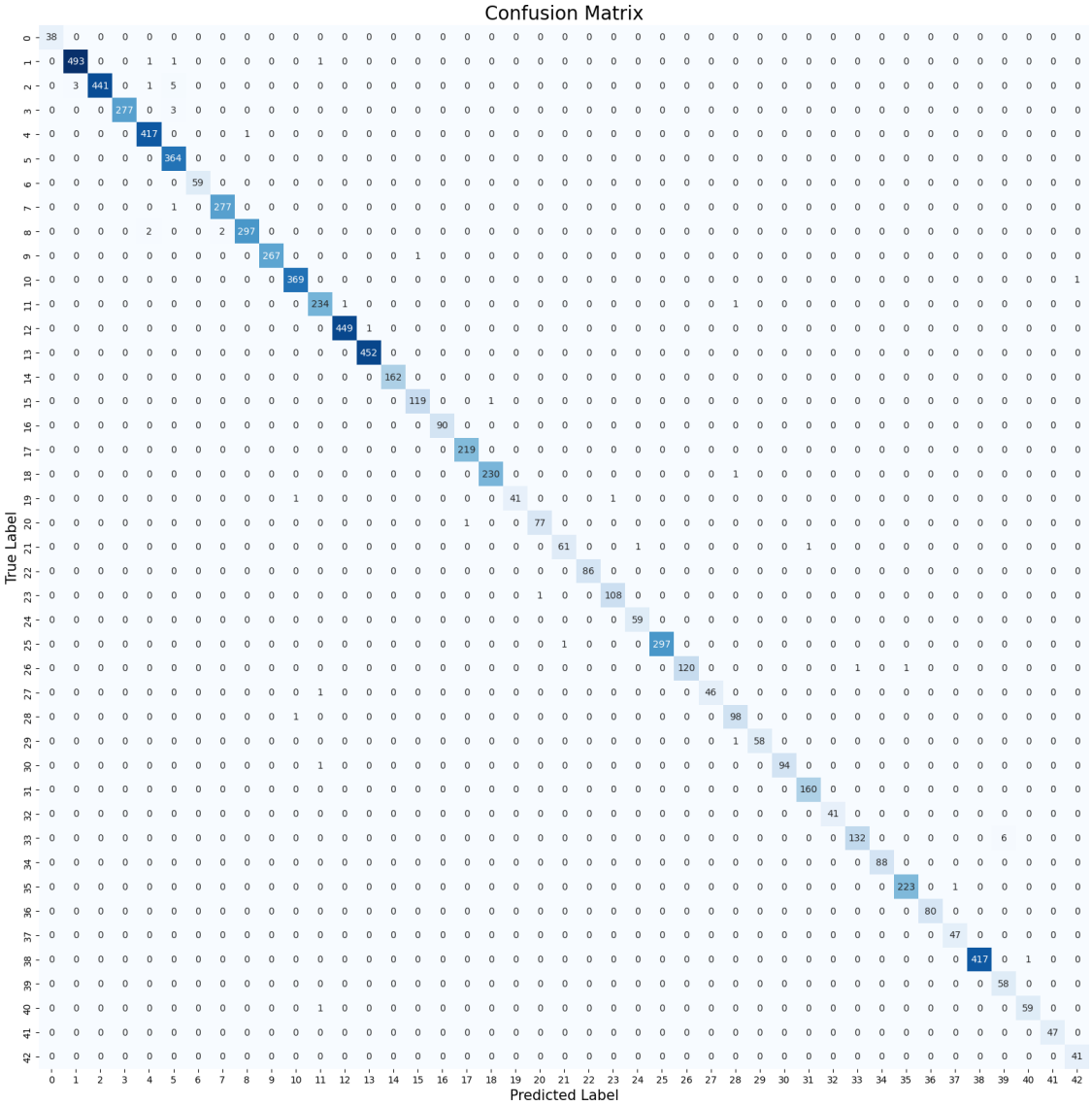
۱.۶ ماتریس درهم‌ریختگی و گزارش طبقه‌بندی

برای محاسبه ماتریس درهم‌ریختگی از کد زیر استفاده شده است:

```
from sklearn.metrics import confusion_matrix, classification_report
pred_probabilities = model.predict(X_val)
pred_classes = np.argmax(pred_probabilities, axis=1)
true_classes = np.argmax(y_val, axis=1)

cm = confusion_matrix(true_classes, pred_classes)
```

```python
# رسم و ذخیره
plt.figure(figsize=(20,20))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.savefig('confusion_matrix.png', bbox_inches='tight')
print(classification_report(true_classes, pred_classes))
```



شکل ۷: ماتریس درهم‌ریختگی

جدول ۱: داده‌های اعتبارسنجی

Support	F1-Score	Recall	Precision	کلاس
38	1.00	1.00	1.00	0
496	0.99	0.99	0.99	1
450	0.99	0.98	1.00	2
280	0.99	0.99	1.00	3
418	0.99	1.00	0.99	4
364	0.99	1.00	0.97	5
59	1.00	1.00	1.00	6
⋮	⋮	⋮	⋮	⋮
41	0.99	1.00	0.98	42
	0.99			Accuracy
	0.99			avg Macro
	0.99			avg Weighted

۲.۶ تحلیل Classification Report

نتایج نشان می‌دهد که مدل CNN در تشخیص تمام 43 کلاس تابلوهای راهنمایی عملکرد بسیار خوبی دارد:

- **Precision** تقریبًا برای تمام کلاس‌ها بالای 97.0 است، یعنی مدل به ندرت پیش‌بینی اشتباه انجام داده است.
- **Recall** برای اکثر کلاس‌ها بالای 98.0 است، به این معنی که مدل تقریبًا تمام نمونه‌های واقعی هر کلاس را شناسایی کرده است.
- **F1-Score** نیز نزدیک به 00.1 است که نشان می‌دهد تعادل خوبی بین Precision و Recall برقرار است.
- **Accuracy کلی** شبکه برابر 99.0 است، که بیانگر دقت بسیار بالا روی داده‌های اعتبارسنجی می‌باشد.
- **avg Macro** و **avg Weighted** نیز هر دو 99.0 هستند، بنابراین مدل عملکردی یکنواخت در تمام کلاس‌ها دارد.

جدول ۲: نام کلاس‌های دیتاست

کد	نام	کد	نام
0	Speed limit (20km/h)	22	Bumpy road
1	Speed limit (30km/h)	23	Slippery road
2	Speed limit (50km/h)	24	Road narrows on the right
3	Speed limit (60km/h)	25	Road work
4	Speed limit (70km/h)	26	Traffic signals
5	Speed limit (80km/h)	27	Pedestrians
6	End of speed limit (80km/h)	28	Children crossing
7	Speed limit (100km/h)	29	Bicycles crossing
8	Speed limit (120km/h)	30	Beware of ice/snow
9	No passing	31	Wild animals crossing
10	No passing veh over 5.3 t	32	End speed + passing limits
11	Right-of-way at intersection	33	Turn right ahead
12	Priority road	34	Turn left ahead
13	Yield	35	Ahead only
14	Stop	36	Go straight or right
15	No vehicles	37	Go straight or left
16	Veh > 5.3 tons prohibited	38	Keep right
17	No entry	39	Keep left
18	General caution	40	Roundabout mandatory
19	Dangerous curve left	41	End of no passing
20	Dangerous curve right	42	End no passing veh > 5.3 t
21	Double curve		

۷ نمونه تست یک تصویر واقعی

۱.۷ محدودیت‌ها

مهم

مدل ارائه‌شده تنها برای تصاویر از پیش برش‌خورده تابلوهای راهنمایی طراحی شده است. در تصاویر واقعی که شامل پس‌زمینه پیچیده، تغییرات نور و مقیاس هستند، به دلیل اختلاف دامنه داده‌ها، عملکرد مدل کاهش می‌یابد.

برای تست روی یک تصویر واقعی از تابع زیر استفاده شده است:

```
def predict_traffic_sign(image_path):
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img, (30,30))
    img_array = np.expand_dims(img_resized/255.0, axis=0)
    pred = model.predict(img_array)
    class_index = np.argmax(pred)
    prob = np.max(pred)
    plt.imshow(img)
```

```
    plt.title(f"Prediction: {classes[class_index]} - {prob*100:.2f}%")
    plt.axis('off')
    plt.show()

# فراخوانی
predict_traffic_sign("test.png")
```

نمونه های خروجی : تصویر ورودی همراه با کد کلاس پیش‌بینی‌شده و میزان اطمینان (Confidence)

(ا) تصویر ورودی

(ب) نتیجه پیش‌بینی مدل

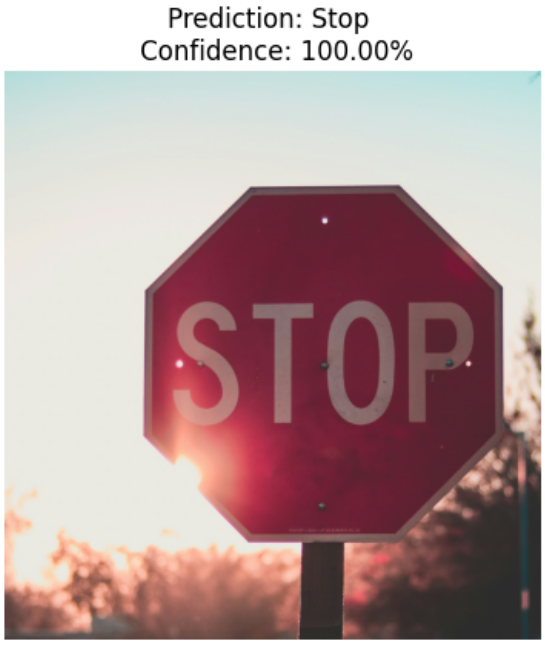
شکل ۸: تست شماره1

(ا) تصویر ورودی

(ب) نتیجه پیش‌بینی مدل

شکل ۹: تست شماره2

(ا) تصویر ورودی



(ب) نتیجه پیش‌بینی مدل

شکل ۱۰: تست شماره3

(ا) تصویر ورودی

(ب) نتیجه پیش‌بینی مدل

شکل ۱۱: تست شماره4

۲.۷ دسترسی به پروژه

پیاده‌سازی کامل پروژه به‌صورت یک نوت‌بوک در Google Colab انجام شده و در لینک زیر ارائه شده است:

Google Colab

References

[ ] Stallkamp, Johannes, et al. *Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition.* Neural Networks, 32 (2012): 323–332.

[ ] GTSRB: German Traffic Sign Recognition Benchmark. http://benchmark.ini.rub.de/?section=gtsrb&subsection=dataset

[ ] Sermanet, Pierre, and Yann LeCun. *Traffic sign recognition with multi-scale convolutional networks.* International Joint Conference on Neural Networks (IJCNN), IEEE, 2011.

[ ] Kaggle Dataset: GTSRB - German Traffic Sign. https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign

[ ] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep Learning.* MIT Press, 2016.